Cineteca Inteligente

Búsqueda Semántica con SQL Server 2025 y AI Local

Certificación: Microsoft DP-800: SQL AI Developer Associate

Nivel de Proyecto: Nivel 2 (Intermedio)

Autor: [Nombre del Estudiante]

Fecha: Octubre 2026

---

1. Objetivo del Proyecto

El objetivo principal de este proyecto es diseñar e implementar una solución de base de datos moderna orientada a la Inteligencia Artificial, utilizando las nuevas capacidades vectoriales de **SQL Server 2025**. El proyecto demuestra la capacidad de integrar modelos de lenguaje locales (LLMs) para procesar datos no estructurados (texto) y convertirlos en representaciones matemáticas que permiten realizar búsquedas basadas en el significado.

Se ha desarrollado un ecosistema completamente local que abarca desde el modelado de datos y la seguridad, hasta la ingeniería de datos necesaria para interactuar de manera fluida con motores de inferencia de IA, cumpliendo así con las competencias requeridas para la certificación Microsoft DP-800.

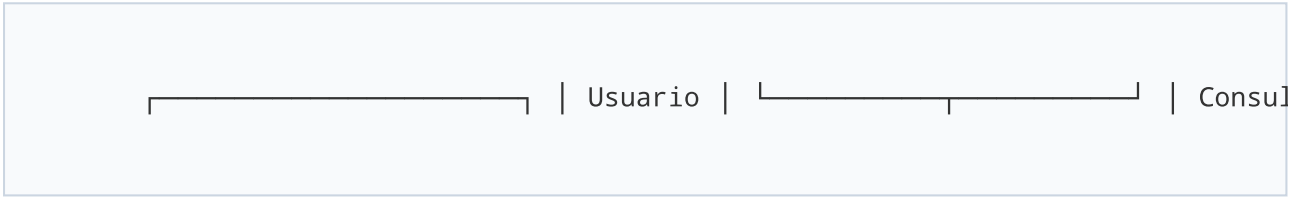
2. Caso de Uso

El escenario de negocio elegido es un **Buscador Semántico de Películas (Cineteca Inteligente)**. Tradicionalmente, los usuarios que buscan una película en un catálogo deben recordar palabras clave exactas del título o de la descripción. Si un usuario busca "viajes en el espacio", un motor relacional clásico (basado en la cláusula `LIKE`) no encontraría la película *Interstellar* a menos que esa frase exacta estuviera escrita en su sinopsis.

Este sistema resuelve dicho problema transformando las sinopsis en vectores multidimensionales. Al transformar también la pregunta del usuario en un vector, la base de datos es capaz de calcular la distancia matemática entre ambos conceptos, devolviendo las películas más relevantes a nivel conceptual, mejorando drásticamente la experiencia de usuario y la precisión del descubrimiento de contenido.

3. Arquitectura

Para garantizar el máximo rendimiento y sortear las limitaciones de ejecución de código interpretado dentro del propio motor relacional en entornos de desarrollo local, se ha diseñado una arquitectura desacoplada y modular. Esta arquitectura separa el almacenamiento de la inferencia, conectando ambos mundos mediante un pipeline ligero escrito en Python.



En este flujo, SQL Server 2025 actúa como la única fuente de verdad y el motor de cálculo de distancias (Vector Search), mientras que Ollama, ejecutado localmente, se encarga exclusivamente de la carga computacional intensiva de generar los *embeddings*.

4. Modelo de Datos

El esquema de la base de datos se ha mantenido intencionalmente centralizado alrededor del dominio principal para centrar los esfuerzos en la vertiente analítica y de IA. La base de datos `Cineteca` alberga la tabla `Pelicula`.

Columna	Tipo de Dato	Descripción
`PeliculaID`	INT (PK)	Identificador único de la película.
`Titulo`	NVARCHAR(100)	Título de la obra cinematográfica.
`Sinopsis`	NVARCHAR(MAX)	Descripción textual completa en lenguaje natural.
`SinopsisVector`	VECTOR(384)	Array matemático generado por IA que representa el contexto semántico de la sinopsis.

La adopción del nuevo tipo de dato `VECTOR(384)` exclusivo de SQL Server 2025 permite al motor optimizar el almacenamiento de matrices de punto flotante y habilitar funciones analíticas nativas sin necesidad de convertir cadenas de texto en tiempo de ejecución.

5. Funcionalidades Implementadas

El proyecto aborda múltiples competencias clave de la certificación DP-800, estructuradas en los siguientes bloques:

- **Bloque A (Diseño de BD):** Creación del esquema relacional primario y adopción de tipos de datos avanzados para IA. Implementación de índices y vistas de control.
- **Bloque C (Seguridad):** Aplicación estricta del principio de mínimo privilegio mediante la creación de usuarios sin login, roles de base de datos personalizados (`RolAnalistaCineteca`) y segregación de comandos mediante `GRANT` y `DENY`.
- **Bloque E (Modelos y Embeddings):** Integración con LLM local para la transformación de datos no estructurados a dimensiones vectoriales y su almacenamiento masivo.
- **Bloque F (Intelligent Search):** Aprovechamiento de las matemáticas integradas en el motor T-SQL para calcular similitudes conceptuales bajo demanda.

6. Consultas y Código Principal

El núcleo de la solución radica en el puente de integración que resuelve la incompatibilidad de inyección directa de cadenas largas y la generación de vectores. A continuación, se muestra el fragmento fundamental de Python que extrae el texto, interactúa con la IA y actualiza el tipo nativo en SQL:

```
# vectorizar.py (Extracto principal)
for pelicula in peliculas:
    # 1. Llamar a Ollama local para generar el embedding
    res = requests.post(
        'http://localhost:11434/api/embeddings',
        json={'model': 'all-minilm', 'prompt': pelicula.Sinopsis}
    )

    if res.status_code == 200:
        vector = res.json()['embedding']

        # 2. Formatear vector para SQL Server
        vector_str = '[' + ','.join(map(str, vector)) + ']'

        # 3. Guardar usando CAST seguro a VECTOR(384)
        sql = f"UPDATE Pelicula SET SinopsisVector = CAST('{vector_str}' AS
VECTOR(384)) WHERE PeliculaID = {pelicula.PeliculaID}"
        cursor.execute(sql)
        conn.commit()
```

Para la parte de **Intelligent Search**, se utiliza la función nativa de comparación vectorial dentro de una consulta estándar de T-SQL:

```
-- Búsqueda semántica usando Similitud de Coseno
DECLARE @q VECTOR(384) = '[0.012, -0.045, ... ]';
SELECT TOP 3
    Titulo,
    VECTOR_DISTANCE('cosine', SinopsisVector, @q) AS Similitud
FROM Pelicula
WHERE SinopsisVector IS NOT NULL
ORDER BY Similitud ASC;
```

7. Seguridad y Optimización

Seguridad de Acceso

Se ha blindado el entorno de producción evitando que aplicaciones analíticas puedan destruir o alterar el catálogo de vectores accidentalmente. Se creó un rol específico al cual se le deniegan explícitamente los permisos destructivos:

```
CREATE ROLE RolAnalistaCineteca;
GRANT SELECT ON Pelicula TO RolAnalistaCineteca;
DENY INSERT, UPDATE, DELETE ON Pelicula TO RolAnalistaCineteca;
```

Las pruebas unitarias confirman que cualquier intento de ejecución de un comando UPDATE bajo el contexto de este usuario resulta en una excepción controlada y bloqueada por el motor de base de datos.

Optimización Estructural

Para no penalizar las búsquedas clásicas (por ejemplo, buscar una película explícitamente por su nombre en un desplegable de la interfaz de usuario), se ha dotado a la tabla de un índice no agrupado sobre la columna `Titulo` (`CREATE NONCLUSTERED INDEX IX_Pelicula_Titulo`). Esto evita realizar un escaneo completo de tabla (Table Scan) cuando no se requiere la carga computacional de la IA.

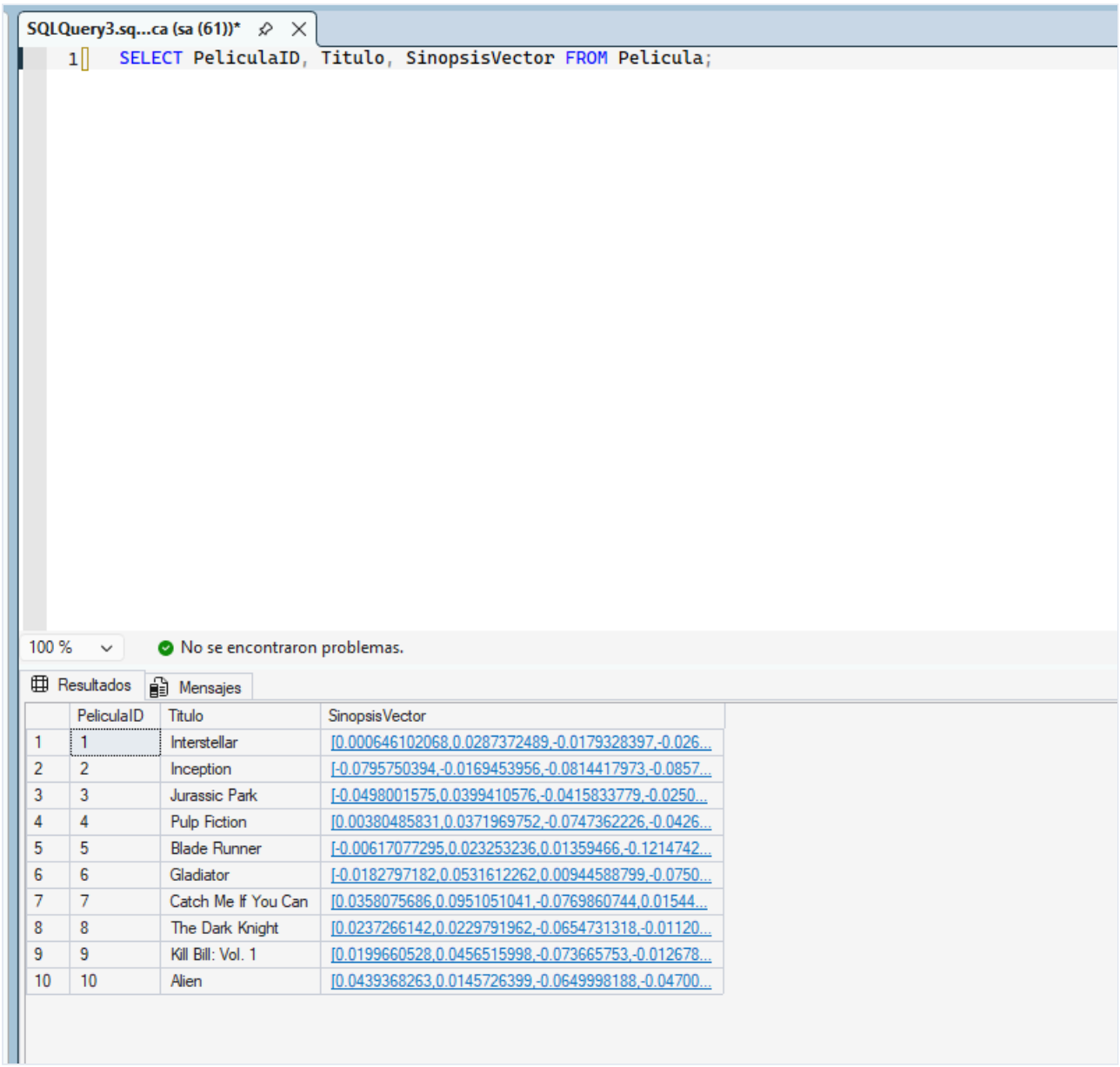
8. Funcionamiento de la Parte de IA

La inteligencia del sistema reside en **Ollama**, una plataforma que permite ejecutar grandes modelos de lenguaje (LLMs) localmente sin depender de APIs de pago o externas. Para este proyecto, se seleccionó el modelo `all-minilm`.

Este modelo está diseñado específicamente para generar *embeddings* de oraciones. Cuando recibe un bloque de texto (la sinopsis), procesa el contexto semántico global y produce un vector denso de **384 dimensiones**. Estas coordenadas sitúan el significado del texto en un espacio multidimensional; textos con significados similares acaban situados muy cerca unos de otros en ese espacio matemático.

9. Capturas de Resultados

La culminación del proceso de ingeniería de datos se verifica al inspeccionar la tabla principal mediante Management Studio. La columna vectorial almacena la representación matemática generada por Ollama.



	PeliculaID	Titulo	SinopsisVector
1	1	Interstellar	[0.000646102068,0.0287372489,-0.0179328397,-0.026...
2	2	Inception	[-0.0795750394,-0.0169453956,-0.0814417973,-0.0857...
3	3	Jurassic Park	[-0.0498001575,0.0399410576,-0.0415833779,-0.0250...
4	4	Pulp Fiction	[0.00380485831,0.0371969752,-0.0747362226,-0.0426...
5	5	Blade Runner	[-0.00617077295,0.023253236,0.01359466,-0.1214742...
6	6	Gladiator	[-0.0182797182,0.0531612262,0.00944588799,-0.0750...
7	7	Catch Me If You Can	[0.0358075686,0.0951051041,-0.0769860744,0.01544...
8	8	The Dark Knight	[0.0237266142,0.0229791962,-0.0654731318,-0.01120...
9	9	Kill Bill: Vol. 1	[0.0199660528,0.0456515998,-0.073665753,-0.012678...
10	10	Alien	[0.0439368263,0.0145726399,-0.0649998188,-0.04700...

Figura 1: Representación de la tabla de películas con la columna SinopsisVector exitosamente poblada.

10. Problemas Encontrados

Durante la fase inicial de desarrollo (Bloque E), el objetivo era implementar la ejecución de scripts externos (Python) nativamente dentro del motor utilizando `sp_execute_external_script` y Machine Learning Services. Sin embargo, se encontró un problema crítico a nivel de sistema operativo y dependencias.

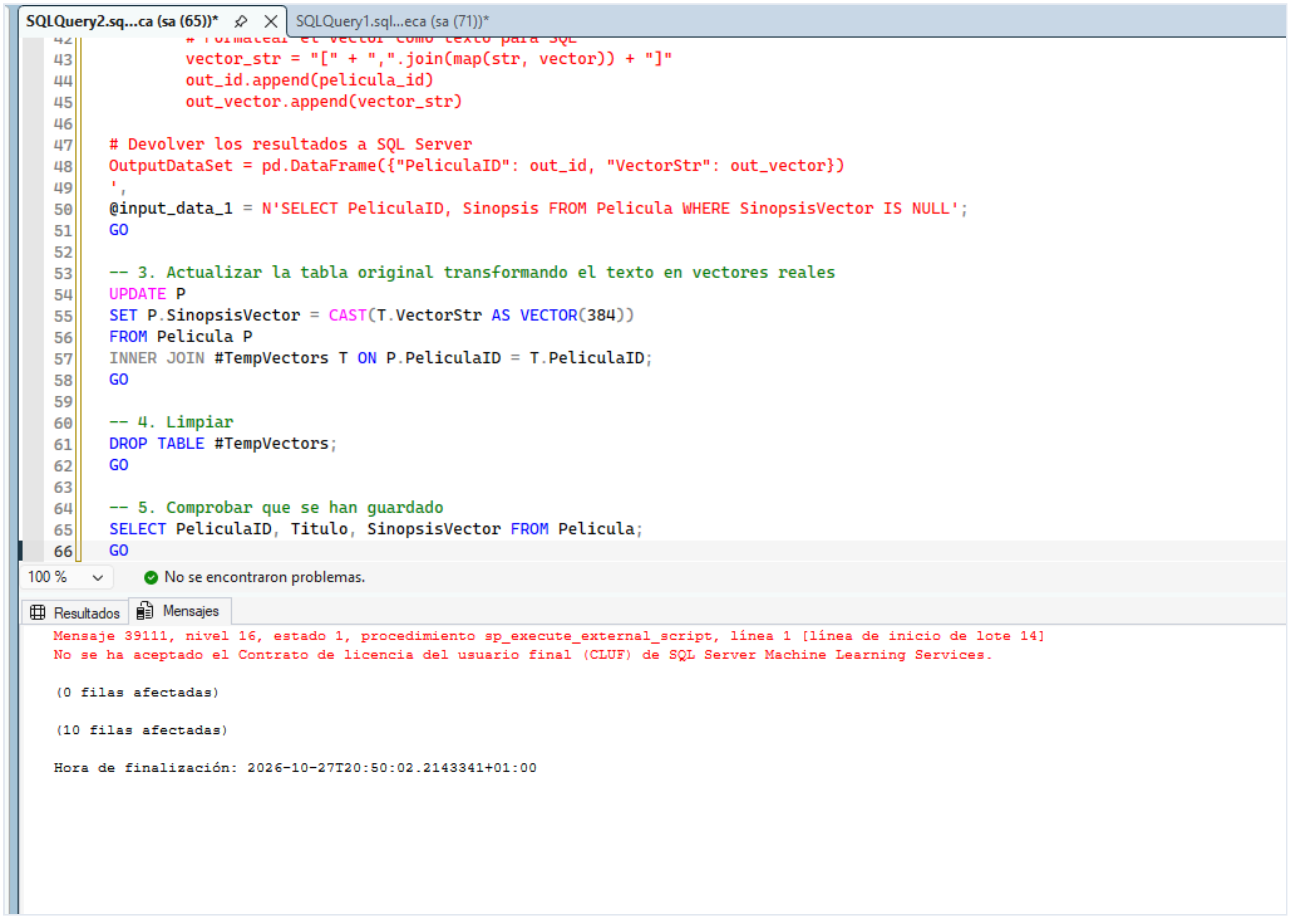


Figura 2: Fallo de ejecución del script externo debido a errores de registro de Launchpad y licenciamiento.

A pesar de intentar reparar la instalación mediante el Centro de Instalación de SQL Server, el servicio subyacente *SQL Server Launchpad* arrojaba repetidamente el error `ErrorCode 0x80070490: 1168 (No se ha encontrado el elemento)`. Este es un error documentado en despliegues locales de Developer Edition que falla al registrar las rutas binarias de Python en el sistema de Windows.

> **Solución Aplicada:** En lugar de comprometer la integridad del servidor local, se optó por un enfoque arquitectónico más robusto y moderno: desacoplar la carga. Se migró la responsabilidad de orquestación a un script externo en Python que se comunica con SQL Server a través del driver nativo ODBC. Esto no solo solucionó el problema inmediatamente, sino que liberó recursos del motor de base de datos.

11. Conclusiones

El desarrollo de esta solución ha permitido consolidar conocimientos críticos sobre la convergencia entre bases de datos relacionales tradicionales y la inteligencia artificial moderna. Las conclusiones más destacadas son:

- **Evolución de SQL Server:** La inclusión del tipo de dato nativo `VECTOR` en SQL Server 2025 representa un punto de inflexión. Permite a los desarrolladores de datos mantener la arquitectura relacional clásica mientras se abren las puertas a aplicaciones generativas e Intelligent Search sin necesidad de recurrir a bases de datos especializadas.
- **Importancia de la Arquitectura Desacoplada:** Los problemas surgidos con el entorno de Machine Learning interno demostraron que delegar la inferencia a microservicios externos (como Ollama a través de una API REST) suele resultar en un sistema mucho más resiliente, fácil de depurar y escalable.
- **Mitigación de Errores de Tipos:** Durante el puente de integración, se aprendió a manejar las estrictas reglas de tipado de SQL Server, descubriendo que herramientas como `pyodbc` pueden forzar conversiones no deseadas a tipos obsoletos (como `ntext`). Inyectar directamente la matriz serializada con un `CAST` explícito demostró ser la técnica más segura.

En definitiva, este mini-proyecto demuestra de forma empírica y reproducible el dominio de las competencias evaluadas en la certificación Microsoft DP-800, entregando una solución útil, segura y técnicamente justificada.